



Serviços e Frameworks para o Livramente

1. Front-end: React

React é uma biblioteca front-end escrita em *JavaScript* para construir interfaces para os usuários. É focado na parte de visualização.

É popular por ser fácil de usar, altamente flexível e escalável.

Razões da escolha:

- Criar aplicações *web* complexas que precisam ser escaláveis e mantidas por equipes grandes;
- Componentes podem ser reutilizados e compartilhados entre diferentes partes da aplicação, tornando o trabalho em grupo mais eficiente;

Requisitos:

- Aprender/entender *JavaScript*;
- Familiaridade com *HTML* e *CSS*;
- Conhecimento de programação funcional.

Links relacionados:

- Site React: [Apresentando react.dev – React](https://react.dev)
- Documentação/Introdução a conceitos do React: [Início rápido – React](#)
- Referência para trabalhar com React: [Visão geral da referência do React](#)

2. Back-end: Node.js

Node.js atua como back-end, processando solicitações de API, interagindo com bancos de dados.

Vantagens: Boa performance em tempo real, grande ecossistemas de pacotes, bom para um MVP rápido, baixa latência e alta performance assíncrona.

Links relacionados:

- Documentação *Node.js*: [Docs | Node.js](#)

NestJS é um framework *Node.js* para desenvolver aplicações back-end eficientes, confiáveis e escaláveis, utilizando TypeScript. O framework é construído por meio de módulos. **NestJS** já vem com estrutura modular. Melhor para times iniciantes, já tem um padrão definido de arquitetura. Bem estruturado.

Express é um framework web para *Node.js* que facilita a criação de aplicações e APIs no lado do servidor. **Express** é leve e direto. Tem controle total, mas precisa criar a arquitetura e manter a organização manualmente.

3. Hospedagem Front-end: Vercel

Vercel é uma plataforma de nuvem para desenvolvedores front-end, auxiliando na hospedagem de sites e aplicações *web*, focado em velocidade e confiança para os desenvolvedores.

Links relacionados:

- Site do Vercel: [Vercel: Build and deploy](#)

4. Hospedagem Back-end: render

Render tem suporte a *Node.js*, deploy via Git. Permite processos em background no plano gratuito (não colocam servidor em espera quando fica inativo por um tempo).

Links relacionados:

- Deploy gratuito no Render: [Deploy for Free – Render Docs](#)

5. Banco de dados: MongoDB

MongoDB é um sistema de gerenciamento de banco de dados (SGBD) NoSQL, de código aberto, que armazena dados em um formato de documento flexível (BSON, uma versão binária de JSON) em vez de tabelas e linhas, como em um banco de dados relacional tradicional. É ideal para aplicações modernas que

exigem alta disponibilidade, escalabilidade e um modelo de dados que evolua com a aplicação.

Links relacionados:

- Site do MongoDB: [MongoDB Atlas: Cloud Document Database | MongoDB](#)
- Documentação MongoDB: [Manual do banco de dados - MongoDB](#)

6. Hospedagem BD: MongoDB Atlas

MongoDB Atlas é uma plataforma de dados multi-nuvem, que simplifica e acelera o desenvolvimento de aplicações ao oferecer um serviço de banco de dados completamente gerenciado. A plataforma tem plano gratuito com 512MB de armazenamento.

Links relacionados:

- Planos do MongoDB Atlas: [Pricing | MongoDB](#)

7. ODM: Mongoose

ODM é uma ferramenta que permite que você interaja com banco de dados não-relacional (MongoDB) usando objetos da linguagem de programação (JavaScript).

Links relacionados:

- Site Mongoose: [Mongoose ODM v8.18.0](#)
- Documentação Mongoose: [Mongoose v8.18.0: Schemas](#)

8. CI/CD: GitHub Actions

CI/CD (Integração Contínua/Entrega Contínua) são práticas DevOps que automatizam a integração de código, testes e implementações para criar um software de forma mais rápida e confiável.

GitHub Actions é uma plataforma de CI/CD integrada diretamente no GitHub, que permite automatizar o desenvolvimento de software com testes e deploy automáticos diretamente no repositório do GitHub.

Links relacionados:

- Documentação GitHub Actions: [GitHub Docs](#)

9. Versionamento: Git

Git é um sistema de controle de versão distribuído, para rastrear e gerenciar alterações no código fonte, mantendo um histórico detalhado de todas as mudanças, permitindo que múltiplos desenvolvedores colaborem em um projeto por meio das possibilidades de voltar a versões anteriores e integrar suas alterações de forma segura e eficiente, evitando conflitos.

Links relacionados:

- Documentação do Git: [Git - Reference](#)

10. Arquitetura back-end: Monolítica

Arquitetura Monolítica é um modelo de desenvolvimento de software onde a aplicação é construída e executada como uma unidade única que executa múltiplas funções.

Razões para a escolha:

- Projeto está em fase inicial (não tem usuários);
- Microserviços são complexos, sendo necessário monitorar centenas de processos, transações distribuídas;
- Início com monolito e migra-se para microserviços em caso de problemas de desempenhos, atraso de *releases*, etc.

Links relacionados:

- Diferença Monolito x microserviços: [Monolítico x microserviços](#)
- Quando não usar microserviços: [Engenharia de Software Moderna](#)

[Uma comparação rápida de frameworks Node JS vs React JS | UXPin](#)